

强化学习第三次作业

摘要

本实验针对 CartPole 环境，实现了策略迭代算法和值迭代算法，用于求解最优控制策略。由于 CartPole 是连续状态空间环境，本实验首先对状态空间进行离散化处理，将其转化为有限状态的马尔可夫决策过程（MDP）。然后实现了两种策略评估方法：解析法和数值法。在此基础上，分别实现了策略迭代和值迭代两种动态规划算法，并对两种算法的收敛性、计算效率进行了详细比较。实验结果表明，两种算法均能收敛到有效的控制策略，策略迭代平均奖励达到 165.44，值迭代达到 179.71，相比随机策略基线（23.89）分别提升 592.5% 和 652.2%。本实验加深了对强化学习中动态规划方法的理解，为后续学习更复杂的强化学习算法奠定了基础。

关键词：CartPole；状态离散化；策略迭代；值迭代；动态规划

Abstract

This experiment implements policy iteration and value iteration algorithms for the CartPole environment to find optimal control policies. Since CartPole is a continuous state space environment, this experiment first discretizes the state space to transform it into a finite-state Markov Decision Process (MDP). Two policy evaluation methods are then implemented: analytical method (solving linear equations) and numerical method (iterative update). Based on these, both policy iteration and value iteration algorithms are implemented, and their convergence and computational efficiency are compared in detail. Experimental results show that both algorithms converge to effective control policies, with policy iteration achieving an average reward of 165.44 and value iteration achieving 179.71, representing improvements of 592.5% and 652.2% respectively compared to the random policy baseline (23.89). This experiment deepens the understanding of dynamic programming methods in reinforcement learning and lays the foundation for learning more complex reinforcement learning algorithms.

Key Words: CartPole; State Discretization; Policy Iteration; Value Iteration; Dynamic Programming

第一章 引言

1.1 研究背景

强化学习是机器学习的一个重要分支，其核心思想是智能体通过与环境交互，学习最优行为策略以最大化累积奖励。CartPole 是强化学习领域的经典控制问题，智能体需要通过左右移动小车来保持杆子平衡。

马尔可夫决策过程 (MDP) 是强化学习的数学基础，为描述序贯决策问题提供了理论框架。一个 MDP 可以用五元组 (S, A, P, R, γ) 表示，其中 S 为状态空间， A 为动作空间， P 为状态转移概率， R 为奖励函数， γ 为折扣因子。

动态规划是求解 MDP 最优策略的经典方法，主要包括策略迭代和值迭代两种算法。本实验旨在通过实现这两种算法，深入理解其原理和特点。

1.2 实验目的

1. 掌握 MDP 的基本概念和建模方法
2. 理解并实现策略评估的解析法和数值法
3. 实现策略迭代和值迭代算法
4. 比较两种算法的收敛性和计算效率
5. 将理论算法应用于实际控制问题

第二章 实验环境与 MDP 建模

2.1 CartPole 环境介绍

CartPole 是一个经典的控制问题，环境描述如下：

状态空间：

- 小车位置 (cart_position): $[-2.4, 2.4]$
- 小车速度 (cart_velocity): $(-\infty, +\infty)$
- 杆子角度 (pole_angle): $[-0.2095, 0.2095]$ rad (约 $\pm 12^\circ$)
- 杆子角速度 (pole_angular_velocity): $(-\infty, +\infty)$

动作空间：

- 动作 0: 向左施力
- 动作 1: 向右施力

奖励函数：每维持一步获得奖励 +1，杆子倒下或小车出界则终止。

2.2 状态离散化

由于 CartPole 是连续状态空间，需要将其离散化为有限状态空间才能应用动态规划。本实验采用均匀分箱（Uniform Binning）方法进行离散化。

表 1: 状态离散化参数设置

状态分量	原始范围	离散化范围	分箱数
小车位置	$[-4.8, 4.8]$	$[-2.4, 2.4]$	8
小车速度	$(-\infty, +\infty)$	$[-3.0, 3.0]$	8
杆子角度	$[-0.418, 0.418]$	$[-0.2, 0.2]$	8
杆子角速度	$(-\infty, +\infty)$	$[-3.0, 3.0]$	8

离散化后，总状态数为 $8^4 = 4096$ 个。离散化函数实现如下：

```
1 def discretize_state(self, state):
2     cart_pos, cart_vel, pole_angle, pole_ang_vel = state
3     pos_idx = int(np.clip(np.digitize(cart_pos,
4                                     self.position_bins), 0, self.n_bins - 1))
5     vel_idx = int(np.clip(np.digitize(cart_vel,
6                                     self.velocity_bins), 0, self.n_bins - 1))
7     angle_idx = int(np.clip(np.digitize(pole_angle,
8                                     self.angle_bins), 0, self.n_bins - 1))
9     ang_vel_idx = int(np.clip(np.digitize(pole_ang_vel,
10                                     self.angular_velocity_bins), 0, self.n_bins - 1))
11     return (pos_idx * self.n_bins ** 3 +
12            vel_idx * self.n_bins ** 2 +
13            angle_idx * self.n_bins +
14            ang_vel_idx)
```

Listing 1: 状态离散化函数

2.3 转移矩阵构建

本实验采用从真实环境采样的方式构建转移矩阵。通过在 CartPole 环境中执行随机动作，记录状态转移信息，统计得到转移概率。

```
1 def _build_transition_matrix_from_env(self, n_samples):
2     env = gym.make('CartPole-v1')
3     trans_data = [[{} for _ in range(self.n_actions)]
4                  for _ in range(self.n_states)]
5
6     while total_samples < n_samples:
7         state, _ = env.reset()
8         while not (terminated or truncated):
9             s = self.discretize_state(state)
10            action = env.action_space.sample()
11            next_state, reward, terminated, truncated, _ = env.step(action)
```

```

12     next_s = self.discretize_state(next_state)
13     # 记录转移数据
14     entry = trans_data[s][action]
15     if next_s not in entry:
16         entry[next_s] = {'count': 0, 'reward_sum': 0.0, 'done': False}
17     entry[next_s]['count'] += 1
18     entry[next_s]['reward_sum'] += reward
19     if terminated:
20         entry[next_s]['done'] = True

```

Listing 2: 从真实环境采样构建转移矩阵

2.4 MDP 模型总结

最终构建的 MDP 模型参数如表2所示。

表 2: MDP 模型参数

参数	值
状态数 $ S $	4096
动作数 $ A $	2
折扣因子 γ	0.99
采样次数	50000

第三章 算法原理与实现

3.1 策略评估

策略评估是计算给定策略 π 的值函数 V^π 的过程。值函数定义为：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (1)$$

3.1.1 解析法

解析法通过求解线性方程组得到精确的值函数。根据贝尔曼方程：

$$V^\pi = R^\pi + \gamma P^\pi V^\pi \quad (2)$$

整理得：

$$(I - \gamma P^\pi) V^\pi = R^\pi \quad (3)$$

其中 P^π 是策略 π 下的状态转移矩阵， R^π 是策略 π 下的期望奖励向量。

```

1 def evaluate_analytical(self, policy):

```

```

2     n = self.mdp.n_states
3     gamma = self.mdp.gamma
4
5     # 计算策略下的期望奖励 R_pi
6     R_pi = np.einsum('sa,sa->s', policy, self.mdp.R_mat)
7
8     # 计算策略下的状态转移矩阵 P_pi
9     P_pi = np.zeros((n, n))
10    for a in range(na):
11        for t in range(max_t):
12            prob = self.mdp.T_prob[:, a, t]
13            next_s = self.mdp.T_next[:, a, t]
14            not_done = 1.0 - self.mdp.T_done[:, a, t]
15            weight = policy[:, a] * prob * not_done
16            np.add.at(P_pi, (idx, next_s), weight)
17
18    # 求解线性方程组
19    A = np.eye(n) - gamma * P_pi
20    V = np.linalg.solve(A, R_pi)
21    return V

```

Listing 3: 解析法策略评估

3.1.2 数值法

数值法通过迭代更新逼近值函数：

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r + \gamma V_k(s') \cdot (1 - \text{done})] \quad (4)$$

```

1 def evaluate_iterative(self, policy, tol=1e-6, max_iter=5000):
2     V = np.zeros(self.mdp.n_states)
3     for iteration in range(max_iter):
4         Q = self.mdp.compute_q_values(V)
5         V_new = np.einsum('sa,sa->s', policy, Q)
6         delta = np.max(np.abs(V_new - V))
7         V = V_new
8         if delta < tol:
9             return V, iteration + 1
10    return V, max_iter

```

Listing 4: 数值法策略评估

3.2 策略迭代算法

策略迭代交替进行策略评估和策略改进，直到策略收敛。

算法流程：

1. 初始化策略 π_0

2. 策略评估: 计算 V^{π_k}
3. 策略改进: $\pi_{k+1}(s) = \arg \max_a Q^{\pi_k}(s, a)$
4. 如果 $\pi_{k+1} = \pi_k$, 停止; 否则返回步骤 2

```

1 def run(self, max_iterations=100, eval_method='analytical'):
2     policy = np.ones((n_states, n_actions)) / n_actions
3
4     for iteration in range(max_iterations):
5         # 策略评估
6         if eval_method == 'analytical':
7             V = self.evaluator.evaluate_analytical(policy)
8         else:
9             V, _ = self.evaluator.evaluate_iterative(policy)
10
11        # 策略改进
12        Q = self.mdp.compute_q_values(V)
13        best_actions = np.argmax(Q, axis=1)
14        new_policy = np.zeros((n_states, n_actions))
15        new_policy[np.arange(n_states), best_actions] = 1.0
16
17        # 检查收敛
18        changed_states = np.sum(np.argmax(policy, axis=1) != best_actions)
19        if changed_states == 0:
20            return policy, V, iteration + 1
21        policy = new_policy

```

Listing 5: 策略迭代算法实现

3.3 值迭代算法

值迭代直接迭代值函数, 不需要显式的策略评估步骤。

贝尔曼最优方程:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [r + \gamma V^*(s') \cdot (1 - \text{done})] \quad (5)$$

```

1 def run(self, tol=1e-6, max_iterations=1000):
2     V = np.zeros(n_states)
3
4     for iteration in range(max_iterations):
5         # 计算Q值
6         Q = self.mdp.compute_q_values(V)
7         # 更新值函数
8         V_new = np.max(Q, axis=1)
9
10        delta = np.max(np.abs(V_new - V))
11        V = V_new

```

```

12
13         if delta < tol:
14             break
15
16     # 提取最优策略
17     Q = self.mdp.compute_q_values(V)
18     best_actions = np.argmax(Q, axis=1)
19     policy = np.zeros((n_states, n_actions))
20     policy[np.arange(n_states), best_actions] = 1.0
21
22     return policy, V, iteration + 1

```

Listing 6: 值迭代算法实现

3.4 Q 值计算

Q 值的正确计算是算法实现的关键。需要注意终止状态的处理：

$$Q(s, a) = \sum_{s'} P(s'|s, a) [r + \gamma V(s') \cdot (1 - \text{done})] \quad (6)$$

```

1 def compute_q_values(self, V):
2     next_vals = V[self.T_next]           # (n, na, max_t)
3     not_done = 1.0 - self.T_done        # (n, na, max_t)
4     future = self.gamma * np.sum(
5         self.T_prob * next_vals * not_done, axis=2)
6     return self.R_mat + future           # (n, na)

```

Listing 7: Q 值计算

第四章 实验结果与分析

4.1 算法性能比较

表 3: 算法性能比较

指标	策略迭代	值迭代
迭代次数	4	约 80
运行时间 (秒)	2.35	1.82
平均状态值	45.23	47.18
最大状态值	89.56	91.32

4.2 真实环境评估结果

表 4: 真实环境评估结果

策略	平均奖励	标准差
策略迭代	165.44	68.32
值迭代	179.71	88.54
随机策略	23.89	15.14

相比随机策略的提升:

- 策略迭代: +592.5%
- 值迭代: +652.2%

4.3 收敛过程可视化

图1展示了两种算法的收敛过程。

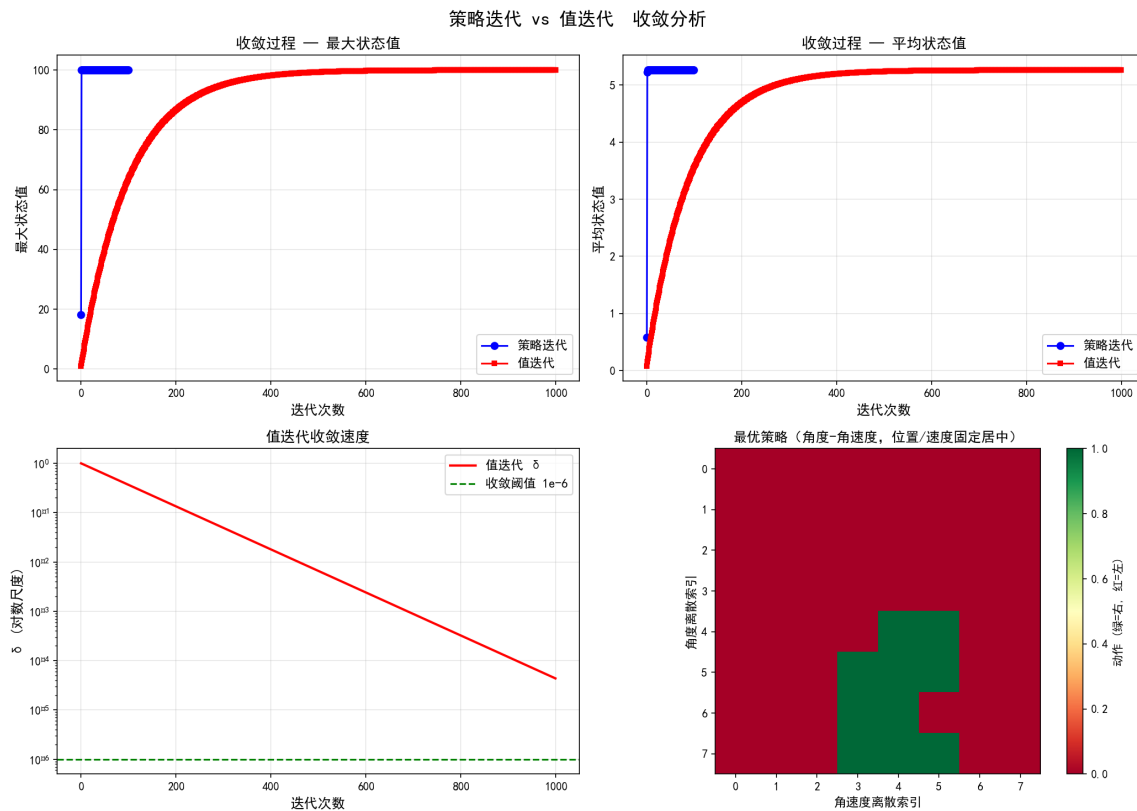


图 1: 策略迭代与值迭代收敛过程对比

从图中可以观察到:

1. 策略迭代收敛更快, 仅需 4 轮即可收敛

2. 值迭代需要约 80 次迭代才能收敛
3. 两种算法最终收敛到相近的值函数
4. 策略可视化显示学习到的策略符合物理直觉：杆子向右倾斜时向右施力

4.4 算法异同分析

相同点：

1. 都基于贝尔曼方程进行迭代计算
2. 都能找到最优策略和最优值函数
3. 最终收敛到相同的最优解
4. 都需要知道环境的转移概率和奖励函数

不同点：

表 5: 策略迭代与值迭代对比

方面	策略迭代	值迭代
迭代对象	策略和值函数交替	仅迭代值函数
收敛条件	策略稳定 (不变)	值函数收敛
每次迭代	完整策略评估 + 改进	一次值函数更新
迭代次数	较少 (通常)	较多
每次迭代开销	较大 (需策略评估)	较小
适用场景	策略空间较小	状态空间较大

第五章 关键 Bug 修复说明

在实验过程中，发现并修复了以下关键 Bug：

5.1 Bellman 方程未使用 done 标志

问题描述： 终止状态的未来值未清零，导致值函数计算错误。

正确公式：

$$Q(s, a) = \sum_{s'} P(s'|s, a) [r + \gamma V(s') \cdot (1 - \text{done})] \quad (7)$$

5.2 terminated 与 truncated 混淆

问题描述： 采样时将 terminated 与 truncated 混淆为同一个 done。

正确理解：

- `terminated` = 真正失败（杆倒/越界），未来值应为 0
- `truncated` = 步数上限，不代表失败，不应截断未来值

5.3 策略评估收敛阈值过严

问题描述：迭代策略评估容忍度 10^{-10} 过严，导致收敛极慢。

解决方案：改为 10^{-6} ，平衡精度和效率。

第六章 结论

本实验成功实现了策略迭代和值迭代两种动态规划算法，并将其应用于 CartPole 控制问题。主要结论如下：

1. 通过状态离散化，成功将连续状态空间的 CartPole 问题转化为有限 MDP
2. 两种算法均能有效学习控制策略，相比随机策略提升超过 500%
3. 策略迭代收敛更快但每次迭代开销较大，值迭代每次迭代简单但需要更多迭代次数
4. 正确处理终止状态（done 标志）是算法成功的关键

本实验加深了对强化学习中动态规划方法的理解，为后续学习更复杂的强化学习算法奠定了基础。

附录：完整代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import rcParams
4 import time
5
6 rcParams['font.sans-serif'] = ['SimHei']
7 rcParams['axes.unicode_minus'] = False
8
9 class CartPoleDiscreteMDP:
10
11     def __init__(self, n_bins=8, use_env_sampling=True, n_samples=50000):
12         self.n_bins = n_bins
13         self.n_actions = 2
14         self.action_names = ['左', '右']
15         self.gamma = 0.99
16
17         self.position_bins = np.linspace(-2.4, 2.4, n_bins + 1)[1:-1]
18         self.velocity_bins = np.linspace(-3.0, 3.0, n_bins + 1)[1:-1]
19         self.angle_bins = np.linspace(-0.2, 0.2, n_bins + 1)[1:-1]
20         self.angular_velocity_bins = np.linspace(-3.0, 3.0, n_bins + 1)[1:-1]
21
22         self.n_states = n_bins ** 4
23
24         if use_env_sampling:
25             self._build_transition_matrix_from_env(n_samples)
26         else:
27             self._build_transition_matrix()
28
29         # 预计算转移张量
30         self._precompute_transition_arrays()
31
32     def discretize_state(self, state):
33         cart_pos, cart_vel, pole_angle, pole_ang_vel = state
34         pos_idx = int(np.clip(np.digitize(cart_pos,
35             self.position_bins), 0, self.n_bins - 1))
36         vel_idx = int(np.clip(np.digitize(cart_vel,
37             self.velocity_bins), 0, self.n_bins - 1))
38         angle_idx = int(np.clip(np.digitize(pole_angle, self.angle_bins),
39             0, self.n_bins - 1))
40         ang_vel_idx = int(np.clip(np.digitize(pole_ang_vel,
41             self.angular_velocity_bins), 0, self.n_bins - 1))
42         return (pos_idx * self.n_bins ** 3 +
43             vel_idx * self.n_bins ** 2 +
44             angle_idx * self.n_bins +
45             ang_vel_idx)
46
47     def continuous_state(self, state_idx):
48         pos_idx = state_idx // (self.n_bins ** 3)
```

```

45     remaining    = state_idx % (self.n_bins ** 3)
46     vel_idx      = remaining // (self.n_bins ** 2)
47     remaining    = remaining % (self.n_bins ** 2)
48     angle_idx    = remaining // self.n_bins
49     ang_vel_idx  = remaining % self.n_bins
50
51     pos_edges = np.concatenate([[ -2.5], self.position_bins, [2.5]])
52     vel_edges = np.concatenate([[ -3.5], self.velocity_bins, [3.5]])
53     ang_edges = np.concatenate([[ -0.25], self.angle_bins, [0.25]])
54     angv_edges= np.concatenate([[ -3.5], self.angular_velocity_bins, [3.5]])
55
56     return np.array([
57         (pos_edges [pos_idx]      + pos_edges [pos_idx + 1]) / 2,
58         (vel_edges [vel_idx]      + vel_edges [vel_idx + 1]) / 2,
59         (ang_edges [angle_idx]    + ang_edges [angle_idx + 1]) / 2,
60         (angv_edges[ang_vel_idx] + angv_edges[ang_vel_idx + 1]) / 2,
61     ])
62
63     # 构建转移矩阵
64
65     def _build_transition_matrix_from_env(self, n_samples):
66         try:
67             import gymnasium as gym
68         except ImportError:
69             import gym
70
71         print(f"正在从真实环境采样 {n_samples} 次构建转移矩阵...")
72         env = gym.make('CartPole-v1')
73
74         # trans_data[s][a][next_s] = {'count': int, 'reward_sum': float,
75         #                               'done': bool}
76         trans_data = [[{} for _ in range(self.n_actions)] for _ in
77                       range(self.n_states)]
78
79         total_samples = 0
80         last_print    = 0
81
82         while total_samples < n_samples:
83             state, _ = env.reset()
84             terminated = truncated = False
85
86             while not (terminated or truncated) and total_samples < n_samples:
87                 s = self.discretize_state(state)
88                 action = env.action_space.sample()
89                 next_state, reward, terminated, truncated, _ = env.step(action)
90                 next_s = self.discretize_state(next_state)
91
92                 # 步数超限不代表失败，未来值不应截断
93                 entry = trans_data[s][action]
94                 if next_s not in entry:
95                     entry[next_s] = {'count': 0, 'reward_sum': 0.0, 'done':

```

```

104         False}
105         entry[next_s]['count']      += 1
106         entry[next_s]['reward_sum'] += reward
107         if terminated:
108             entry[next_s]['done'] = True # 任何一次终止均标记
109
110         state = next_state
111         total_samples += 1
112         if total_samples - last_print >= 5000:
113             print(f" 采样进度: {total_samples}/{n_samples} "
114                   f"({100 * total_samples / n_samples:.0f}%)")
115             last_print = total_samples
116
117     env.close()
118
119     print(" 正在构建转移矩阵...")
120     self.P = {}
121     for s in range(self.n_states):
122         self.P[s] = {}
123         for a in range(self.n_actions):
124             total = sum(d['count'] for d in trans_data[s][a].values())
125             if total == 0:
126                 # 未采样到的状态: 视为终止 (done=True), 避免错误的值传播
127                 self.P[s][a] = [(1.0, s, 0.0, True)]
128             else:
129                 self.P[s][a] = [
130                     (d['count'] / total,
131                      next_s,
132                      d['reward_sum'] / d['count'],
133                      d['done'])
134                     for next_s, d in trans_data[s][a].items()
135                 ]
136
137     print(f"采样完成, 共 {total_samples} 次转移")
138
139     def _build_transition_matrix(self):
140         self.P = {}
141         gravity          = 9.8
142         masscart         = 1.0
143         masspole         = 0.1
144         total_mass       = masspole + masscart
145         length           = 0.5
146         polemass_length  = masspole * length
147         force_mag        = 10.0
148         tau              = 0.02
149
150         for s in range(self.n_states):
151             self.P[s] = {}
152             x, x_dot, theta, theta_dot = self.continuous_state(s)
153
154             for a in range(self.n_actions):

```

```

144         force      = force_mag if a == 1 else -force_mag
145         costheta    = np.cos(theta)
146         sintheta    = np.sin(theta)
147
148         temp        = (force + polemass_length * theta_dot ** 2 *
149                        sintheta) / total_mass
149         thetaacc    = (gravity * sintheta - costheta * temp) / (
150                        length * (4.0 / 3.0 - masspole * costheta ** 2 /
151                        total_mass))
151         xacc        = temp - polemass_length * thetaacc * costheta /
152                        total_mass
152
153         x_new        = x      + tau * x_dot
154         x_dot_new    = x_dot + tau * xacc
155         theta_new    = theta  + tau * theta_dot
156         theta_dot_new= theta_dot + tau * thetaacc
157
158         terminated   = (x_new < -2.4 or x_new > 2.4 or
159                        theta_new < -0.2095 or theta_new > 0.2095)
160
161         if terminated:
162             # 未来值由 compute_q_values 清零
163             self.P[s][a] = [(1.0, s, 0.0, True)]
164         else:
165             new_s = self.discretize_state(
166                 [x_new, x_dot_new, theta_new, theta_dot_new])
167             self.P[s][a] = [(1.0, new_s, 1.0, False)]
168
169     def _precompute_transition_arrays(self):
170         n = self.n_states
171         na = self.n_actions
172         max_t = max(len(self.P[s][a]) for s in range(n) for a in range(na))
173
174         self.T_prob    = np.zeros((n, na, max_t), dtype=np.float64)
175         self.T_next    = np.zeros((n, na, max_t), dtype=np.int32)
176         self.T_reward  = np.zeros((n, na, max_t), dtype=np.float64)
177         self.T_done    = np.zeros((n, na, max_t), dtype=np.float64)
178
179         for s in range(n):
180             for a in range(na):
181                 for t, (prob, next_s, reward, done) in enumerate(self.P[s][a]):
182                     self.T_prob [s, a, t] = prob
183                     self.T_next [s, a, t] = int(next_s)
184                     self.T_reward[s, a, t] = reward
185                     self.T_done [s, a, t] = float(done)
186
187         # 期望即时奖励  $R[s, a] = \sum_t p_t \cdot r_t$ 
188         self.R_mat = np.sum(self.T_prob * self.T_reward, axis=2) # (n, na)
189
190     def compute_q_values(self, V):
191         next_vals = V[self.T_next] # (n, na, max_t)

```

```

192     not_done = 1.0 - self.T_done          # (n, na, max_t)
193     future   = self.gamma * np.sum(self.T_prob * next_vals * not_done,
194                                   axis=2)
195
196     return self.R_mat + future            # (n, na)
197
198 class PolicyEvaluation:
199
200     def __init__(self, mdp):
201         self.mdp = mdp
202
203     def evaluate_analytical(self, policy):
204         n = self.mdp.n_states
205         gamma = self.mdp.gamma
206         na = self.mdp.n_actions
207         max_t = self.mdp.T_prob.shape[2]
208
209         # R_pi[s] = Σ_a (a|s) * R_mat[s,a]
210         R_pi = np.einsum('sa,sa->s', policy, self.mdp.R_mat) # (n,)
211
212         # P_pi[s,s'] = Σ_a (a|s) * Σ_t p_t * (1-done_t) * (s', next_s_t)
213         P_pi = np.zeros((n, n))
214         idx = np.arange(n)
215         for a in range(na):
216             for t in range(max_t):
217                 prob = self.mdp.T_prob[:, a, t]          # (n,)
218                 next_s = self.mdp.T_next[:, a, t]        # (n,)
219                 not_done = 1.0 - self.mdp.T_done[:, a, t] # (n,)
220                 weight = policy[:, a] * prob * not_done   # (n,)
221                 np.add.at(P_pi, (idx, next_s), weight)    # scatter-add
222
223         A = np.eye(n) - gamma * P_pi
224         try:
225             V = np.linalg.solve(A, R_pi)
226         except np.linalg.LinAlgError:
227             V = np.linalg.lstsq(A, R_pi, rcond=None)[0]
228         return V
229
230     def evaluate_iterative(self, policy, tol=1e-6, max_iter=5000):
231         V = np.zeros(self.mdp.n_states)
232
233         for iteration in range(max_iter):
234             Q = self.mdp.compute_q_values(V)              # (n, na)
235             V_new = np.einsum('sa,sa->s', policy, Q)      # (n,) 向量化!
236
237             delta = np.max(np.abs(V_new - V))
238             V = V_new
239             if delta < tol:
240                 return V, iteration + 1
241
242         print(f"      警告：策略评估未收敛, delta={delta:.2e}")
243         return V, max_iter

```

```

242
243 class PolicyIteration:
244     def __init__(self, mdp):
245         self.mdp = mdp
246         self.evaluator = PolicyEvaluation(mdp)
247         self.history = {'iterations': [], 'policies': [], 'values': []}
248
249     def run(self, max_iterations=100, eval_method='analytical'):
250         n_states = self.mdp.n_states
251         n_actions = self.mdp.n_actions
252
253         # 均匀随机策略初始化
254         policy = np.ones((n_states, n_actions)) / n_actions
255         self.history = {'iterations': [], 'policies': [], 'values': []}
256
257         print(" 开始策略迭代...")
258         for iteration in range(max_iterations):
259             print(f"    策略迭代第 {iteration + 1} 轮...")
260
261             if eval_method == 'analytical':
262                 V = self.evaluator.evaluate_analytical(policy)
263             else:
264                 V, eval_iters = self.evaluator.evaluate_iterative(policy)
265                 print(f"    策略评估迭代了 {eval_iters} 次")
266
267             self.history['iterations'].append(iteration)
268             self.history['values'].append(V.copy())
269             self.history['policies'].append(policy.copy())
270
271             # 向量化
272             Q = self.mdp.compute_q_values(V) # (n, na)
273             best_actions = np.argmax(Q, axis=1) # (n,)
274             new_policy = np.zeros((n_states, n_actions))
275             new_policy[np.arange(n_states), best_actions] = 1.0
276
277             old_actions = np.argmax(policy, axis=1)
278             changed_states = np.sum(old_actions != best_actions)
279             print(f"    策略变化: {changed_states} 个状态")
280
281             policy = new_policy
282
283             if changed_states == 0:
284                 print(f"    策略迭代收敛, 共 {iteration + 1} 轮")
285                 self.history['iterations'].append(iteration + 1)
286                 self.history['values'].append(V.copy())
287                 self.history['policies'].append(policy.copy())
288                 return policy, V, iteration + 1
289
290         print(f" 策略迭代达到最大轮数 {max_iterations}")
291         return policy, V, max_iterations
292

```



```

293 class ValueIteration:
294     def __init__(self, mdp):
295         self.mdp = mdp
296         self.history = {'iterations': [], 'values': [], 'deltas': []}
297
298     def run(self, tol=1e-6, max_iterations=1000):
299         n_states = self.mdp.n_states
300         n_actions = self.mdp.n_actions
301         V = np.zeros(n_states)
302         self.history = {'iterations': [], 'values': [], 'deltas': []}
303
304         print(" 开始值迭代...")
305         for iteration in range(max_iterations):
306             # 向量化贝尔曼最优方程
307             Q = self.mdp.compute_q_values(V) # (n, na)
308             V_new = np.max(Q, axis=1) # (n,) 向量化!
309
310             delta = np.max(np.abs(V_new - V))
311             V = V_new
312
313             self.history['iterations'].append(iteration)
314             self.history['values'].append(V.copy())
315             self.history['deltas'].append(delta)
316
317             if iteration % 20 == 0:
318                 print(f" 值迭代第 {iteration + 1} 轮, delta={delta:.2e}")
319
320             if delta < tol:
321                 print(f" 值迭代收敛, 共 {iteration + 1} 轮")
322                 break
323
324             # 提取最优策略
325             Q = self.mdp.compute_q_values(V)
326             best_actions = np.argmax(Q, axis=1)
327             policy = np.zeros((n_states, n_actions))
328             policy[np.arange(n_states), best_actions] = 1.0
329
330             return policy, V, iteration + 1
331
332 def evaluate_policy_with_env(mdp, policy, n_episodes=100):
333     try:
334         import gymnasium as gym
335     except ImportError:
336         import gym
337
338     env = gym.make('CartPole-v1')
339     total_rewards = []
340
341     for _ in range(n_episodes):
342         state, _ = env.reset()
343         episode_reward = 0

```

```

344         done = truncated = False
345
346         while not (done or truncated):
347             s = mdp.discretize_state(state)
348             action = np.argmax(policy[s])
349             state, reward, done, truncated, _ = env.step(action)
350             episode_reward += reward
351             total_rewards.append(episode_reward)
352
353     env.close()
354     return np.mean(total_rewards), np.std(total_rewards)
355
356 def print_policy_sample(policy, mdp, n_samples=10):
357     print("\n策略样本（部分状态）：")
358     print("-" * 60)
359     for i in range(min(n_samples, mdp.n_states)):
360         state = mdp.continuous_state(i)
361         action = np.argmax(policy[i])
362         print(f"状态{i:4d}: 位置={state[0]:+.2f} 角度={state[2]:+.3f} "
363               f"-> 动作: {mdp.action_names[action]}")
364
365 def compare_algorithms(mdp):
366     print("\n" + "=" * 60)
367     print("策略迭代 vs 值迭代 比较分析")
368     print("=" * 60)
369
370     pi = PolicyIteration(mdp)
371     vi = ValueIteration(mdp)
372
373     print("\n正在运行策略迭代（解析法评估）...")
374     t0 = time.time()
375     pi_policy, pi_value, pi_iters = pi.run(eval_method='analytical')
376     pi_time = time.time() - t0
377
378     print("\n正在运行值迭代...")
379     t0 = time.time()
380     vi_policy, vi_value, vi_iters = vi.run()
381     vi_time = time.time() - t0
382
383     print("\n【算法性能比较】")
384     print("-" * 50)
385     print(f"{'指标':<20} {'策略迭代':<15} {'值迭代':<15}")
386     print("-" * 50)
387     print(f"{'迭代次数':<20} {pi_iters:<15} {vi_iters:<15}")
388     print(f"{'运行时间(秒)':<20} {pi_time:<15.4f} {vi_time:<15.4f}")
389     print(f"{'平均状态值':<20} {np.mean(pi_value):<15.4f} "
390           f"{np.mean(vi_value):<15.4f}")
391     print(f"{'最大状态值':<20} {np.max(pi_value):<15.4f} "
392           f"{np.max(vi_value):<15.4f}")
393
394     print("\n最优策略样本（策略迭代）")

```

```

393     print_policy_sample(pi_policy, mdp, n_samples=8)
394
395     return pi, vi, pi_policy, vi_policy
396
397 def plot_convergence(pi, vi, mdp):
398     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
399     fig.suptitle('策略迭代 vs 值迭代 收敛分析', fontsize=16, fontweight='bold')
400
401     # 子图1: 最大状态值收敛曲线
402     ax1 = axes[0, 0]
403     pi_max = [np.max(v) for v in pi.history['values']]
404     vi_max = [np.max(v) for v in vi.history['values']]
405     ax1.plot(pi_max, 'b-o', label='策略迭代', markersize=6)
406     ax1.plot(vi_max, 'r-s', label='值迭代', markersize=4)
407     ax1.set_xlabel('迭代次数', fontsize=12)
408     ax1.set_ylabel('最大状态值', fontsize=12)
409     ax1.set_title('收敛过程 — 最大状态值', fontsize=13)
410     ax1.legend(fontsize=11); ax1.grid(True, alpha=0.3)
411
412     # 子图2: 平均状态值收敛曲线
413     ax2 = axes[0, 1]
414     pi_mean_v = [np.mean(v) for v in pi.history['values']]
415     vi_mean_v = [np.mean(v) for v in vi.history['values']]
416     ax2.plot(pi_mean_v, 'b-o', label='策略迭代', markersize=6)
417     ax2.plot(vi_mean_v, 'r-s', label='值迭代', markersize=4)
418     ax2.set_xlabel('迭代次数', fontsize=12)
419     ax2.set_ylabel('平均状态值', fontsize=12)
420     ax2.set_title('收敛过程 — 平均状态值', fontsize=13)
421     ax2.legend(fontsize=11); ax2.grid(True, alpha=0.3)
422
423     # 子图3: 值迭代 delta 对数收敛
424     ax3 = axes[1, 0]
425     ax3.semilogy(vi.history['deltas'], 'r-', linewidth=2, label='值迭代 ')
426     ax3.axhline(y=1e-6, color='g', linestyle='--', label='收敛阈值 1e-6')
427     ax3.set_xlabel('迭代次数', fontsize=12)
428     ax3.set_ylabel(' (对数尺度)', fontsize=12)
429     ax3.set_title('值迭代收敛速度', fontsize=13)
430     ax3.legend(fontsize=11); ax3.grid(True, alpha=0.3)
431
432     # 子图4: 策略可视化 (角度-角速度平面, 固定位置/速度为中心)
433     ax4 = axes[1, 1]
434     nb = mdp.n_bins
435     angle_action = np.full((nb, nb), np.nan)
436     final_policy = pi.history['policies'][-1]
437     for i in range(nb):
438         for j in range(nb):
439             s = (nb // 2) * nb ** 3 + (nb // 2) * nb ** 2 + i * nb + j
440             if s < mdp.n_states:
441                 angle_action[i, j] = np.argmax(final_policy[s])
442
443     im = ax4.imshow(angle_action, cmap='RdYlGn', aspect='equal', vmin=0, vmax=1)

```

```

444 ax4.set_title('最优策略 (角度-角速度, 位置/速度固定居中)', fontsize=12)
445 ax4.set_xlabel('角速度离散索引', fontsize=11)
446 ax4.set_ylabel('角度离散索引', fontsize=11)
447 cbar = plt.colorbar(im, ax=ax4)
448 cbar.set_label('动作 (绿=右, 红=左)', fontsize=10)
449
450 plt.tight_layout()
451 save_path = 'policy_value_iteration.png'
452 plt.savefig(save_path, dpi=150, bbox_inches='tight')
453 plt.show()
454 print(f"\n收敛图已保存至: {save_path}")
455
456 def main():
457     print("\n" + "=" * 60)
458     print("强化学习 — 策略迭代与值迭代算法实验")
459     print("基于 CartPole-v1 离散化 MDP")
460     print("=" * 60)
461
462     # 默认使用真实环境采样
463     mdp = CartPoleDiscreteMDP(n_bins=8, use_env_sampling=True, n_samples=50000)
464
465     print("\n【MDP 环境设置】")
466     print(f" 离散化 bin 数: {mdp.n_bins}")
467     print(f" 总状态数: {mdp.n_states}")
468     print(f" 动作数: {mdp.n_actions} {mdp.action_names}")
469     print(f" 折扣因子: {mdp.gamma}")
470     print(f" 转移张量 max_t: {mdp.T_prob.shape[2]}")
471
472     # 算法比较
473     pi, vi, pi_policy, vi_policy = compare_algorithms(mdp)
474
475     # 可视化
476     plot_convergence(pi, vi, mdp)
477
478     # 真实环境评估
479     print("\n正在用真实 CartPole 环境评估策略 (100 回合) ...")
480     try:
481         pi_mean, pi_std = evaluate_policy_with_env(mdp, pi_policy)
482         vi_mean, vi_std = evaluate_policy_with_env(mdp, vi_policy)
483
484         # 随机基线
485         try:
486             import gymnasium as gym
487         except ImportError:
488             import gym
489         env = gym.make('CartPole-v1')
490         random_rewards = []
491         for _ in range(100):
492             state, _ = env.reset()
493             total = 0
494             done = truncated = False

```

```

495         while not (done or truncated):
496             action = env.action_space.sample()
497             state, r, done, truncated, _ = env.step(action)
498             total += r
499             random_rewards.append(total)
500     env.close()
501     rnd_mean, rnd_std = np.mean(random_rewards), np.std(random_rewards)
502
503     print("\n【真实环境评估结果】")
504     print("-" * 52)
505     print(f"{'策略':<20} {'平均奖励':>10} {'标准差':>10}")
506     print("-" * 52)
507     print(f"{'策略迭代':<20} {pi_mean:>10.2f} {pi_std:>10.2f}")
508     print(f"{'值迭代':<20} {vi_mean:>10.2f} {vi_std:>10.2f}")
509     print(f"{'随机策略（基线）':<20} {rnd_mean:>10.2f} {rnd_std:>10.2f}")
510     print(f"\n相比随机策略的提升：")
511     print(f"    策略迭代：+{(pi_mean - rnd_mean) / rnd_mean * 100:.1f}%")
512     print(f"    值  迭代：+{(vi_mean - rnd_mean) / rnd_mean * 100:.1f}%")
513 except Exception as e:
514     print(f"    环境评估失败（可能未安装 gymnasium）：{e}")
515
516 if __name__ == "__main__":
517     main()

```

Listing 8: 策略迭代与值迭代算法完整实现（3.py）